

Chapter 1

Algorithms with numbers

One of the main themes of this chapter is the dramatic contrast between two ancient problems that at first seem very similar:

Factoring: Given a number N , express it as a product of its prime factors.

Primality: Given a number N , determine whether it is a prime.

Factoring is hard. Despite centuries of effort by some of the world's smartest mathematicians and computer scientists, the fastest methods for factoring a number N take time exponential in the number of bits of N .

On the other hand, we shall soon see that we *can* efficiently test whether N is prime! And (it gets even more interesting) this strange disparity between the two intimately related problems, one very hard and the other very easy, lies at the heart of the technology that enables secure communication in today's global information environment.

En route to these insights, we need to develop algorithms for a variety of computational tasks involving numbers. We begin with basic arithmetic, an especially appropriate starting point because, as we know, the word *algorithms* originally applied only to methods for these problems.

1.1 Basic arithmetic

1.1.1 Addition

We were so young when we learned the standard technique for addition that we would scarcely have thought to ask *why* it works. But let's go back now and take a closer look.

It is a basic property of decimal numbers that

The sum of any three single-digit numbers is at most two digits long.

Quick check: the sum is at most $9 + 9 + 9 = 27$, two digits long. In fact, this rule holds not just in decimal but in *any* base $b \geq 2$ (Exercise 1.1). In binary, for instance, the maximum possible sum of three single-bit numbers is 3, which is a 2-bit number.

This simple rule gives us a way to add two numbers in any base: align their right-hand ends, and then perform a single right-to-left pass in which the sum is computed digit by digit, maintaining the overflow as a carry. Since we know each individual sum is a two-digit

Bases and logs

Naturally, there is nothing special about the number 10—we just happen to have 10 fingers, and so 10 was an obvious place to pause and take counting to the next level. The Mayans developed a similar positional system based on the number 20 (no shoes, see?). And of course today computers represent numbers in binary.

How many digits are needed to represent the number $N \geq 0$ in base b ? Let's see—with k digits in base b we can express numbers up to $b^k - 1$; for instance, in decimal, three digits get us all the way up to $999 = 10^3 - 1$. By solving for k , we find that $\lceil \log_b(N + 1) \rceil$ digits (about $\log_b N$ digits, give or take 1) are needed to write N in base b .

How much does the size of a number change when we change bases? Recall the rule for converting logarithms from base a to base b : $\log_b N = (\log_a N) / (\log_a b)$. So the size of integer N in base a is the same as its size in base b , times a constant factor $\log_a b$. In big- O notation, therefore, the base is irrelevant, and we write the size simply as $O(\log N)$. When we do not specify a base, as we almost never will, we mean $\log_2 N$.

Incidentally, this function $\log N$ appears repeatedly in our subject, in many guises. Here's a sampling:

1. $\log N$ is, of course, the power to which you need to raise 2 in order to obtain N .
2. Going backward, it can also be seen as the number of times you must halve N to get down to 1. (More precisely: $\lceil \log N \rceil$.) This is useful when a number is halved at each iteration of an algorithm, as in several examples later in the chapter.
3. It is the number of bits in the binary representation of N . (More precisely: $\lceil \log(N+1) \rceil$.)
4. It is also the depth of a complete binary tree with N nodes. (More precisely: $\lceil \log N \rceil$.)
5. It is even the sum $1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{N}$, to within a constant factor (Exercise 1.5).

number, *the carry is always a single digit*, and so at any given step, three single-digit numbers are added. Here's an example showing the addition $53 + 35$ in binary.

$$\begin{array}{rcccccc}
 \text{Carry:} & 1 & & 1 & 1 & 1 & \\
 & 1 & 1 & 0 & 1 & 0 & 1 \quad (53) \\
 & 1 & 0 & 0 & 0 & 1 & 1 \quad (35) \\
 \hline
 & 1 & 0 & 1 & 1 & 0 & 0 \quad (88)
 \end{array}$$

Ordinarily we would spell out the algorithm in pseudocode, but in this case it is so familiar that we do not repeat it. Instead we move straight to analyzing its efficiency.

Given two binary numbers x and y , how long does our algorithm take to add them? This is the kind of question we shall persistently be asking throughout this book. We want the answer expressed as a function of *the size of the input*: the number of bits of x and y , the number of keystrokes needed to type them in.

Suppose x and y are each n bits long; in this chapter we will consistently use the letter n for the sizes of numbers. Then the sum of x and y is $n + 1$ bits at most, and each individual bit

Now that we have a working algorithm whose running time we know, our thoughts wander inevitably to the question of whether there is something even better.

Some readers may be confused at this point: Why $O(n)$ operations? Isn't binary addition something that computers today perform by just one instruction? There are two answers. First, it is certainly true that in a single instruction we can add integers whose size in bits is within the *word length* of today's computers—32 perhaps. But, as will become apparent later in this chapter, it is often useful and necessary to handle numbers much larger than this, perhaps several thousand bits long. Adding and multiplying such large numbers on real computers is very much like performing the operations bit by bit. Second, when we want to understand algorithms, it makes sense to study even the basic algorithms that are encoded in the hardware of today's computers. In doing so, we shall focus on the *bit complexity* of the algorithm, the number of elementary operations on individual bits—because this accounting reflects the amount of hardware, transistors and wires, necessary for implementing the algorithm.

Onward to multiplication! The grade-school algorithm for multiplying two numbers x and y is to create an array of intermediate sums, each representing the product of x by a single digit of y . These values are appropriately left-shifted and then added up. Suppose for instance that we want to multiply 13×11 , or in binary notation, $x = 1101$ and $y = 1011$. The multiplication would proceed thus.

In binary this is particularly easy since each intermediate row is either zero or x itself, left-shifted an appropriate amount of times. Also notice that left-shifting is just a quick way to multiply by the base, which in this case is 2. (Likewise, the effect of a right shift is to divide by the base, rounding down if needed.)

23

Figure 1.1 Multiplication à la Français.

function multiply(x, y)

Input: Two n -bit integers x and y , where $y \geq 0$

Output: Their product

```
if  $y = 0$ : return 0
```

```
 $z = \text{multiply}(x, \lfloor y/2 \rfloor)$ 
```

```
if  $y$  is even:
```

```
    return  $2z$ 
```

```
else:
```

```
    return  $x + 2z$ 
```

rows, with lengths of up to $2n$ bits (taking the shifting into account). The total time taken to add up these rows, doing two numbers at a time, is

$$\underbrace{O(n) + O(n) + \cdots + O(n)}_{n-1 \text{ times}},$$

which is $O(n^2)$, *quadratic* in the size of the inputs: still polynomial but much slower than addition (as we have all suspected since elementary school).

But Al Khwarizmi knew another way to multiply, a method which is used today in some European countries. To multiply two decimal numbers x and y , write them next to each other, as in the example below. Then repeat the following: divide the first number by 2, rounding down the result (that is, dropping the .5 if the number was odd), and double the second number. Keep going till the first number gets down to 1. Then strike out all the rows in which the first number is even, and add up whatever remains in the second column.

11	13	
5	26	
2	52	(strike out)
1	104	
143		(answer)

But if we now compare the two algorithms, binary multiplication and multiplication by repeated halvings of the multiplier, we notice that they are doing the same thing! The three numbers added in the second algorithm are precisely the multiples of 13 by powers of 2 that were added in the binary method. Only this time 11 was not given to us explicitly in binary, and so we had to extract its binary representation by looking at the parity of the numbers obtained from it by successive divisions by 2. Al Khwarizmi's second algorithm is a fascinating mixture of decimal and binary!

The same algorithm can thus be repackaged in different ways. For variety we adopt a third formulation, the recursive algorithm of Figure 1.1, which directly implements the rule

$$x \cdot y = \begin{cases} 2(x \cdot \lfloor y/2 \rfloor) & \text{if } y \text{ is even} \\ x + 2(x \cdot \lfloor y/2 \rfloor) & \text{if } y \text{ is odd.} \end{cases}$$

Figure 1.2 Division.

function `divide`(x, y)

Input: Two n -bit integers x and y , where $y \geq 1$

Output: The quotient and remainder of x divided by y

```
if  $x = 0$ : return  $(q, r) = (0, 0)$ 
 $(q, r) = \text{divide}(\lfloor x/2 \rfloor, y)$ 
 $q = 2 \cdot q, r = 2 \cdot r$ 
if  $x$  is odd:  $r = r + 1$ 
if  $r \geq y$ :  $r = r - y, q = q + 1$ 
return  $(q, r)$ 
```

Is this algorithm correct? The preceding recursive rule is transparently correct; so checking the correctness of the algorithm is merely a matter of verifying that it mimics the rule and that it handles the base case ($y = 0$) properly.

How long does the algorithm take? It must terminate after n recursive calls, because at each call y is halved—that is, its number of bits is decreased by one. And each recursive call requires these operations: a division by 2 (right shift); a test for odd/even (looking up the last bit); a multiplication by 2 (left shift); and possibly one addition, a total of $O(n)$ bit operations. The total time taken is thus $O(n^2)$, just as before.

Can we do better? Intuitively, it seems that multiplication requires adding about n multiples of one of the inputs, and we know that each addition is linear, so it would appear that n^2 bit operations are inevitable. Astonishingly, in Chapter 2 we'll see that we *can* do significantly better!

Division is next. To divide an integer x by another integer $y \neq 0$ means to find a quotient q and a remainder r , where $x = yq + r$ and $r < y$. We show the recursive version of division in Figure 1.2; like multiplication, it takes quadratic time. The analysis of this algorithm is the subject of Exercise 1.8.

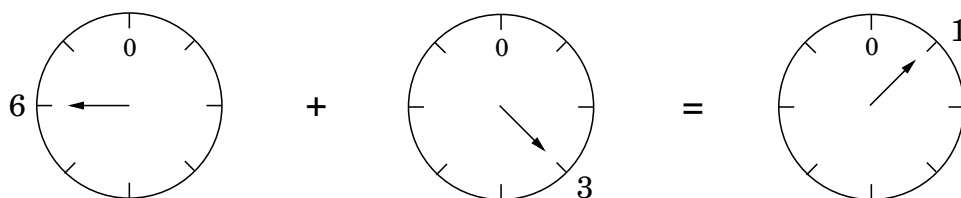
1.2 Modular arithmetic

With repeated addition or multiplication, numbers can get cumbersome large. So it is fortunate that we reset the hour to zero whenever it reaches 24, and the month to January after every stretch of 12 months. Similarly, for the built-in arithmetic operations of computer processors, numbers are restricted to some size, 32 bits say, which is considered generous enough for most purposes.

For the applications we are working toward—primality testing and cryptography—it is necessary to deal with numbers that are significantly larger than 32 bits, but whose range is nonetheless limited.

Modular arithmetic is a system for dealing with restricted ranges of integers. We define x modulo N to be the remainder when x is divided by N ; that is, if $x = qN + r$ with $0 \leq r < N$,

Figure 1.3 Addition modulo 8.



then x modulo N is equal to r . This gives an enhanced notion of equivalence between numbers: x and y are *congruent modulo N* if they differ by a multiple of N , or in symbols,

$$x \equiv y \pmod{N} \iff N \text{ divides } (x - y).$$

For instance, $253 \equiv 13 \pmod{60}$ because $253 - 13$ is a multiple of 60; more familiarly, 253 minutes is 4 hours and 13 minutes. These numbers can also be negative, as in $59 \equiv -1 \pmod{60}$: when it is 59 minutes past the hour, it is also 1 minute short of the next hour.

One way to think of modular arithmetic is that it limits numbers to a predefined range $\{0, 1, \dots, N - 1\}$ and wraps around whenever you try to leave this range—like the hand of a clock (Figure 1.3).

Another interpretation is that modular arithmetic deals with all the integers, but divides them into N *equivalence classes*, each of the form $\{i + kN : k \in \mathbb{Z}\}$ for some i between 0 and $N - 1$. For example, there are three equivalence classes modulo 3:

$$\begin{array}{ccccccccccc} \dots & -9 & -6 & -3 & 0 & 3 & 6 & 9 & \dots \\ \dots & -8 & -5 & -2 & 1 & 4 & 7 & 10 & \dots \\ \dots & -7 & -4 & -1 & 2 & 5 & 8 & 11 & \dots \end{array}$$

Any member of an equivalence class is substitutable for any other; when viewed modulo 3, the numbers 5 and 11 are no different. Under such substitutions, addition and multiplication remain well-defined:

Substitution rule If $x \equiv x' \pmod{N}$ and $y \equiv y' \pmod{N}$, then:

$$x + y \equiv x' + y' \pmod{N} \quad \text{and} \quad xy \equiv x'y' \pmod{N}.$$

(See Exercise 1.9.) For instance, suppose you watch an entire season of your favorite television show in one sitting, starting at midnight. There are 25 episodes, each lasting 3 hours. At what time of day are you done? Answer: the hour of completion is $(25 \times 3) \bmod 24$, which (since $25 \equiv 1 \bmod 24$) is $1 \times 3 = 3 \bmod 24$, or three o'clock in the morning.

It is not hard to check that in modular arithmetic, the usual associative, commutative, and distributive properties of addition and multiplication continue to apply, for instance:

$x + (y + z) \equiv (x + y) + z \pmod{N}$	Associativity
$xy \equiv yx \pmod{N}$	Commutativity
$x(y + z) \equiv xy + yz \pmod{N}$	Distributivity

Taken together with the substitution rule, this implies that while performing a sequence of arithmetic operations, it is legal to reduce intermediate results to their remainders modulo N at any stage. Such simplifications can be a dramatic help in big calculations. Witness, for instance:

$$2^{345} \equiv (2^5)^{69} \equiv 32^{69} \equiv 1^{69} \equiv 1 \pmod{31}.$$

Two's complement

Modular arithmetic is nicely illustrated in *two's complement*, the most common format for storing signed integers. It uses n bits to represent numbers in the range $[-2^{n-1}, 2^{n-1} - 1]$ and is usually described as follows:

- Positive integers, in the range 0 to $2^{n-1} - 1$, are stored in regular binary and have a leading bit of 0.
- Negative integers $-x$, with $1 \leq x \leq 2^{n-1}$, are stored by first constructing x in binary, then flipping all the bits, and finally adding 1. The leading bit in this case is 1.

(And the usual description of addition and multiplication in this format is even more arcane!)

Here's a much simpler way to think about it: any number in the range -2^{n-1} to $2^{n-1} - 1$ is stored modulo 2^n . Negative numbers $-x$ therefore end up as $2^n - x$. Arithmetic operations like addition and subtraction can be performed directly in this format, ignoring any overflow bits that arise.

1.2.1 Modular addition and multiplication

To *add* two numbers x and y modulo N , we start with regular addition. Since x and y are each in the range 0 to $N - 1$, their sum is between 0 and $2(N - 1)$. If the sum exceeds $N - 1$, we merely need to subtract off N to bring it back into the required range. The overall computation therefore consists of an addition, and possibly a subtraction, of numbers that never exceed $2N$. Its running time is linear in the sizes of these numbers, in other words $O(n)$, where $n = \lceil \log N \rceil$ is the size of N ; as a reminder, our convention is to use the letter n to denote input size.

To *multiply* two mod- N numbers x and y , we again just start with regular multiplication and then reduce the answer modulo N . The product can be as large as $(N - 1)^2$, but this is still at most $2n$ bits long since $\log(N - 1)^2 = 2 \log(N - 1) \leq 2n$. To reduce the answer modulo N , we compute the remainder upon dividing it by N , using our quadratic-time division algorithm. Multiplication thus remains a quadratic operation.

Division is not quite so easy. In ordinary arithmetic there is just one tricky case—division by zero. It turns out that in modular arithmetic there are potentially other such cases as well, which we will characterize toward the end of this section. Whenever division is legal, however, it can be managed in cubic time, $O(n^3)$.

To complete the suite of modular arithmetic primitives we need for cryptography, we next turn to *modular exponentiation*, and then to the *greatest common divisor*, which is the key to

Figure 1.4 Modular exponentiation.

function modexp(x, y, N)

Input: Two n -bit integers x and N , an integer exponent y

Output: $x^y \bmod N$

```
if  $y = 0$ : return 1
 $z = \text{modexp}(x, \lfloor y/2 \rfloor, N)$ 
if  $y$  is even:
    return  $z^2 \bmod N$ 
else:
    return  $x \cdot z^2 \bmod N$ 
```

division. For both tasks, the most obvious procedures take exponentially long, but with some ingenuity polynomial-time solutions can be found. A careful choice of algorithm makes all the difference.

1.2.2 Modular exponentiation

In the cryptosystem we are working toward, it is necessary to compute $x^y \bmod N$ for values of x , y , and N that are several hundred bits long. Can this be done quickly?

The result is some number modulo N and is therefore itself a few hundred bits long. However, the raw value of x^y could be much, much longer than this. Even when x and y are just 20-bit numbers, x^y is at least $(2^{19})^{(2^{19})} = 2^{(19)(524288)}$, about 10 million bits long! Imagine what happens if y is a 500-bit number!

To make sure the numbers we are dealing with never grow too large, we need to perform all intermediate computations modulo N . So here's an idea: calculate $x^y \bmod N$ by repeatedly multiplying by x modulo N . The resulting sequence of intermediate products,

$$x \bmod N \rightarrow x^2 \bmod N \rightarrow x^3 \bmod N \rightarrow \cdots \rightarrow x^y \bmod N,$$

consists of numbers that are smaller than N , and so the individual multiplications do not take too long. But there's a problem: if y is 500 bits long, we need to perform $y - 1 \approx 2^{500}$ multiplications! This algorithm is clearly exponential in the size of y .

Luckily, we *can* do better: starting with x and *squaring repeatedly* modulo N , we get

$$x \bmod N \rightarrow x^2 \bmod N \rightarrow x^4 \bmod N \rightarrow x^8 \bmod N \rightarrow \cdots \rightarrow x^{2^{\lfloor \log y \rfloor}} \bmod N.$$

Each takes just $O(\log^2 N)$ time to compute, and in this case there are only $\log y$ multiplications. To determine $x^y \bmod N$, we simply multiply together an appropriate subset of these powers, those corresponding to 1's in the binary representation of y . For instance,

$$x^{25} = x^{11001_2} = x^{10000_2} \cdot x^{1000_2} \cdot x^{12_2} = x^{16} \cdot x^8 \cdot x^1.$$

A polynomial-time algorithm is finally within reach!

Figure 1.5 Euclid’s algorithm for finding the greatest common divisor of two numbers.

```
function Euclid( $a, b$ )  
Input: Two integers  $a$  and  $b$  with  $a \geq b \geq 0$   
Output:  $\text{gcd}(a, b)$   
  
if  $b = 0$ : return  $a$   
return Euclid( $b, a \bmod b$ )
```

We can package this idea in a particularly simple form: the recursive algorithm of Figure 1.4, which works by executing, modulo N , the self-evident rule

$$x^y = \begin{cases} (x^{\lfloor y/2 \rfloor})^2 & \text{if } y \text{ is even} \\ x \cdot (x^{\lfloor y/2 \rfloor})^2 & \text{if } y \text{ is odd.} \end{cases}$$

In doing so, it closely parallels our recursive multiplication algorithm (Figure 1.1). For instance, that algorithm would compute the product $x \cdot 25$ by an analogous decomposition to the one we just saw: $x \cdot 25 = x \cdot 16 + x \cdot 8 + x \cdot 1$. And whereas for multiplication the terms $x \cdot 2^i$ come from repeated *doubling*, for exponentiation the corresponding terms x^{2^i} are generated by repeated squaring.

Let n be the size in bits of x , y , and N (whichever is largest of the three). As with multiplication, the algorithm will halt after at most n recursive calls, and during each call it multiplies n -bit numbers (doing computation modulo N saves us here), for a total running time of $O(n^3)$.

1.2.3 Euclid’s algorithm for greatest common divisor

Our next algorithm was discovered well over 2000 years ago by the mathematician Euclid, in ancient Greece. Given two integers a and b , it finds the largest integer that divides both of them, known as their *greatest common divisor* (gcd).

The most obvious approach is to first factor a and b , and then multiply together their common factors. For instance, $1035 = 3^2 \cdot 5 \cdot 23$ and $759 = 3 \cdot 11 \cdot 23$, so their gcd is $3 \cdot 23 = 69$. However, we have no efficient algorithm for factoring. Is there some other way to compute greatest common divisors?

Euclid’s algorithm uses the following simple formula.

Euclid’s rule *If x and y are positive integers with $x \geq y$, then $\text{gcd}(x, y) = \text{gcd}(x \bmod y, y)$.*

Proof. It is enough to show the slightly simpler rule $\text{gcd}(x, y) = \text{gcd}(x - y, y)$ from which the one stated can be derived by repeatedly subtracting y from x .

Here it goes. Any integer that divides both x and y must also divide $x - y$, so $\text{gcd}(x, y) \leq \text{gcd}(x - y, y)$. Likewise, any integer that divides both $x - y$ and y must also divide both x and y , so $\text{gcd}(x, y) \geq \text{gcd}(x - y, y)$. ■

Euclid’s rule allows us to write down an elegant recursive algorithm (Figure 1.5), and its correctness follows immediately from the rule. In order to figure out its running time, we need to understand how quickly the arguments (a, b) decrease with each successive recursive call. In a single round, arguments (a, b) become $(b, a \bmod b)$: their order is swapped, and the larger of them, a , gets reduced to $a \bmod b$. This is a substantial reduction.

Figure 1.6 A simple extension of Euclid’s algorithm.

function `extended-Euclid`(a, b)

Input: Two positive integers a and b with $a \geq b \geq 0$

Output: Integers x, y, d such that $d = \gcd(a, b)$ and $ax + by = d$

if $b = 0$: return $(1, 0, a)$

$(x', y', d) = \text{Extended-Euclid}(b, a \bmod b)$

return $(y', x' - \lfloor a/b \rfloor y', d)$

Lemma If $a \geq b$, then $a \bmod b < a/2$.

Proof. Witness that either $b \leq a/2$ or $b > a/2$. These two cases are shown in the following figure. If $b \leq a/2$, then we have $a \bmod b < b \leq a/2$; and if $b > a/2$, then $a \bmod b = a - b < a/2$.



■

This means that after any two consecutive rounds, both arguments, a and b , are at the very least halved in value—the length of each decreases by at least one bit. If they are initially n -bit integers, then the base case will be reached within $2n$ recursive calls. And since each call involves a quadratic-time division, the total time is $O(n^3)$.

1.2.4 An extension of Euclid’s algorithm

A small extension to Euclid’s algorithm is the key to dividing in the modular world.

To motivate it, suppose someone claims that d is the greatest common divisor of a and b : how can we check this? It is not enough to verify that d divides both a and b , because this only shows d to be a common factor, not necessarily the largest one. Here’s a test that can be used if d is of a particular form.

Lemma If d divides both a and b , and $d = ax + by$ for some integers x and y , then necessarily $d = \gcd(a, b)$.

Proof. By the first two conditions, d is a common divisor of a and b and so it cannot exceed the greatest common divisor; that is, $d \leq \gcd(a, b)$. On the other hand, since $\gcd(a, b)$ is a common divisor of a and b , it must also divide $ax + by = d$, which implies $\gcd(a, b) \leq d$. Putting these together, $d = \gcd(a, b)$. ■

So, if we can supply two numbers x and y such that $d = ax + by$, then we can be sure $d = \gcd(a, b)$. For instance, we know $\gcd(13, 4) = 1$ because $13 \cdot 1 + 4 \cdot (-3) = 1$. But when can we find these numbers: under what circumstances can $\gcd(a, b)$ be expressed in this checkable form? It turns out that it *always* can. What is even better, the coefficients x and y can be found by a small extension to Euclid’s algorithm; see Figure 1.6.

Lemma For any positive integers a and b , the extended Euclid algorithm returns integers x , y , and d such that $\gcd(a, b) = d = ax + by$.

Proof. The first thing to confirm is that if you ignore the x 's and y 's, the extended algorithm is exactly the same as the original. So, at least we compute $d = \gcd(a, b)$.

For the rest, the recursive nature of the algorithm suggests a proof by induction. The recursion ends when $b = 0$, so it is convenient to do induction on the value of b .

The base case $b = 0$ is easy enough to check directly. Now pick any larger value of b . The algorithm finds $\gcd(a, b)$ by calling $\gcd(b, a \bmod b)$. Since $a \bmod b < b$, we can apply the inductive hypothesis to this recursive call and conclude that the x' and y' it returns are correct:

$$\gcd(b, a \bmod b) = bx' + (a \bmod b)y'.$$

Writing $(a \bmod b)$ as $(a - \lfloor a/b \rfloor b)$, we find

$$d = \gcd(a, b) = \gcd(b, a \bmod b) = bx' + (a \bmod b)y' = bx' + (a - \lfloor a/b \rfloor b)y' = ay' + b(x' - \lfloor a/b \rfloor y').$$

Therefore $d = ax + by$ with $x = y'$ and $y = x' - \lfloor a/b \rfloor y'$, thus validating the algorithm's behavior on input (a, b) . ■

Example. To compute $\gcd(25, 11)$, Euclid's algorithm would proceed as follows:

$$\begin{aligned} 25 &= 2 \cdot \underline{11} + 3 \\ \underline{11} &= 3 \cdot \underline{3} + 2 \\ \underline{3} &= 1 \cdot \underline{2} + 1 \\ \underline{2} &= 2 \cdot \underline{1} + 0 \end{aligned}$$

(at each stage, the gcd computation has been reduced to the underlined numbers). Thus $\gcd(25, 11) = \gcd(11, 3) = \gcd(3, 2) = \gcd(2, 1) = \gcd(1, 0) = 1$.

To find x and y such that $25x + 11y = 1$, we start by expressing 1 in terms of the last pair $(1, 0)$. Then we work backwards and express it in terms of $(2, 1)$, $(3, 2)$, $(11, 3)$, and finally $(25, 11)$. The first step is:

$$1 = \underline{1} - \underline{0}.$$

To rewrite this in terms of $(2, 1)$, we use the substitution $0 = 2 - 2 \cdot 1$ from the last line of the gcd calculation to get:

$$1 = \underline{1} - (\underline{2} - 2 \cdot \underline{1}) = -1 \cdot \underline{2} + 3 \cdot \underline{1}.$$

The second-last line of the gcd calculation tells us that $1 = 3 - 1 \cdot 2$. Substituting:

$$1 = -1 \cdot \underline{2} + 3(\underline{3} - 1 \cdot \underline{2}) = 3 \cdot \underline{3} - 4 \cdot \underline{2}.$$

Continuing in this same way with substitutions $2 = 11 - 3 \cdot 3$ and $3 = 25 - 2 \cdot 11$ gives:

$$1 = 3 \cdot \underline{3} - 4(\underline{11} - 3 \cdot \underline{3}) = -4 \cdot \underline{11} + 15 \cdot \underline{3} = -4 \cdot \underline{11} + 15(\underline{25} - 2 \cdot \underline{11}) = 15 \cdot \underline{25} - 34 \cdot \underline{11}.$$

We're done: $15 \cdot 25 - 34 \cdot 11 = 1$, so $x = 15$ and $y = -34$.

1.2.5 Modular division

In real arithmetic, every number $a \neq 0$ has an inverse, $1/a$, and dividing by a is the same as multiplying by this inverse. In modular arithmetic, we can make a similar definition.

We say x is the *multiplicative inverse* of a modulo N if $ax \equiv 1 \pmod{N}$.

There can be at most one such x modulo N (Exercise 1.23), and we shall denote it by a^{-1} . However, this inverse does not always exist! For instance, 2 is not invertible modulo 6: that is, $2x \not\equiv 1 \pmod{6}$ for every possible choice of x . In this case, a and N are both even and thus then $a \pmod{N}$ is always even, since $a \pmod{N} = a - kN$ for some k . More generally, we can be certain that $\gcd(a, N)$ divides $ax \pmod{N}$, because this latter quantity can be written in the form $ax + kN$. So if $\gcd(a, N) > 1$, then $ax \not\equiv 1 \pmod{N}$, no matter what x might be, and therefore a cannot have a multiplicative inverse modulo N .

In fact, this is the only circumstance in which a is not invertible. When $\gcd(a, N) = 1$ (we say a and N are *relatively prime*), the extended Euclid algorithm gives us integers x and y such that $ax + Ny = 1$, which means that $ax \equiv 1 \pmod{N}$. Thus x is a 's sought inverse.

Example. Continuing with our previous example, suppose we wish to compute $11^{-1} \pmod{25}$. Using the extended Euclid algorithm, we find that $15 \cdot 25 - 34 \cdot 11 = 1$. Reducing both sides modulo 25, we have $-34 \cdot 11 \equiv 1 \pmod{25}$. So $-34 \equiv 16 \pmod{25}$ is the inverse of 11 modulo 25.

Modular division theorem *For any $a \pmod{N}$, a has a multiplicative inverse modulo N if and only if it is relatively prime to N . When this inverse exists, it can be found in time $O(n^3)$ (where as usual n denotes the number of bits of N) by running the extended Euclid algorithm.*

This resolves the issue of modular division: when working modulo N , we can divide by numbers relatively prime to N —and only by these. And to actually carry out the division, we multiply by the inverse.

Is your social security number a prime?

The numbers 7, 17, 19, 71, and 79 are primes, but how about 717-19-7179? Telling whether a reasonably large number is a prime seems tedious because there are far too many candidate factors to try. However, there are some clever tricks to speed up the process. For instance, you can omit even-valued candidates after you have eliminated the number 2. You can actually omit all candidates except those that are themselves primes.

In fact, a little further thought will convince you that you can proclaim N a prime as soon as you have rejected all candidates *up to* $\sqrt{N}$, for if N can indeed be factored as $N = K \cdot L$, then it is impossible for both factors to exceed $\sqrt{N}$.

We seem to be making progress! Perhaps by omitting more and more candidate factors, a truly efficient primality test can be discovered.

Unfortunately, there is no fast primality test down this road. The reason is that we have been trying to tell if a number is a prime *by factoring it*. And factoring is a hard problem!

Modern cryptography, as well as the balance of this chapter, is about the following important idea: **factoring is hard and primality is easy**. We cannot factor large numbers, but we can easily test huge numbers for primality! (Presumably, if a number is composite, such a test will detect this *without finding a factor*.)